



《AI大模型Ragent项目》——AI基础设施层宏观设计

来自： 拿个offer-开源&项目实战



马丁

2026年04月07日 22:35

前面的 RAG 理论系列和知识库实战系列，我们从 RAG 的概念讲到了评估优化，从文件上传讲到了分块存储。到这里，你应该已经清楚一个 RAG 系统的完整链路：用户提问 → 检索知识库 → 重排序 → 喂给大模型生成回答。

但有一个问题一直被我们当作黑盒处理——模型调用。

前面的文章里，每次需要调大模型的时候，我们都是直接写 OkHttp 请求，往百炼或者硅基流动发一个 HTTP 调用，拿到结果就完事。这在教学环境里没问题，但放到 Ragent 这样一个需要同时支持百炼、硅基流动、Ollama 等多个供应商，并且覆盖 Chat、Embedding、Rerank 三种模型能力的项目里，事情就没这么简单了。

这些模型调用在项目到底是怎么组织的？直接在 Service 里写 OkHttp 调用？如果百炼挂了怎么办？如果要加一个新的供应商怎么办？

从这篇开始，我们进入大模型调度引擎实战系列。这个系列会逐篇拆解 Ragent 项目的 infra-ai 模块——它是整个项目的 AI 基础设施层，负责屏蔽供应商差异、实现多模型路由和故障转移。这一篇先从宏观视角讲清楚它的整体设计。

为什么需要 AI 基础设施层

1. 没有 infra 层的世界

假设你在做一个电商客服知识库系统。产品需求很明确：用户提问时，先调 Embedding 模型把问题向量化，再去向量数据库检索相关文档，用 Reranker 精排一遍，最后把 chunk 喂给 Chat 模型生成回答。

一开始，你选了百炼作为模型供应商，直接在业务 Service 里写调用代码：

```
// ChatService.java — 直接调百炼 Chat API
public String chat(String prompt) {
    JsonObject body = new JsonObject();
    body.addProperty("model", "qwen-plus");
    body.add("messages", buildMessages(prompt));

    Request request = new Request.Builder()
        .url("https://dashscope.aliyuncs.com/compatible-mode/v1/chat/completions")
        .addHeader("Authorization", "Bearer " + BAILIAN_API_KEY)
        .post(RequestBody.create(body.toString(), JSON))
        .build();

    Response response = httpClient.newCall(request).execute();
    // ... 解析响应
}
```

能跑，没毛病。Embedding 调用也差不多，换个 URL 和请求格式就行。

然后问题来了。

场景一：百炼 API 挂了。某天下午百炼服务抖了一下，返回 500 错误。你的整个客服系统直接瘫痪——因为所有对话都走百炼，没有备选方案。老板问你为什么系统挂了，你说供应商挂了。老板说那你为什么不做个备份？

场景二：加一个硅基流动做备份。老板的要求合理，你开始加硅基流动。但硅基流动的 API 地址不一样

(<https://api.siliconflow.cn/v1/chat/completions>)，认证方式一样但模型名不同 (deepseek-ai/DeepSeek-V3)。你在 ChatService 里加了一堆 if-else：

```
public String chat(String prompt) {
    if ("bailian".equals(currentProvider)) {
        // 百炼的 URL、Header、模型名
    } else if ("siliconflow".equals(currentProvider)) {
        // 硅基流动的 URL、Header、模型名
    }
    // 失败了还要 try-catch 切换到另一个供应商...
}
```

场景三：再加一个 Ollama 做本地部署。测试环境不想烧钱调云端 API，装了一个 Ollama 跑本地模型。但 Ollama 不需要 API Key，端点路径也不一样 (<http://localhost:11434/v1/chat/completions>)。又是一套 if-else。

场景四：Embedding 和 Rerank 也要同样的逻辑。 你发现 EmbeddingService 也需要做供应商切换和容错，RerankService 也要。三个 Service 里面，每个都有一堆重复的 HTTP 调用代码、供应商判断逻辑、错误处理代码。

到这里，代码已经变成了这个样子：三个 Service 里散落着三个供应商的 if-else 判断，每个 Service 都在重复做 HTTP 请求构建、响应解析、错误处理。供应商的 URL、API Key、模型名硬编码在代码里。想加个新供应商？每个 Service 都得改。想调整优先级？改代码重新部署。

这就是没有基础设施层的代价。

2. 我们需要什么

从上面的反面例子里，可以提炼出四个核心需求：

统一接口，屏蔽差异。 业务层不应该关心当前调的是百炼还是硅基流动，更不应该知道各家 API 的请求格式有什么区别。业务层只需要说我要做一次 Chat 调用，传入 Prompt，拿到结果。

多模型路由与故障转移。 同一种能力（比如 Chat）可以配置多个候选模型，按优先级排序。高优先级的模型挂了，自动切换到下一个，业务层完全无感知。

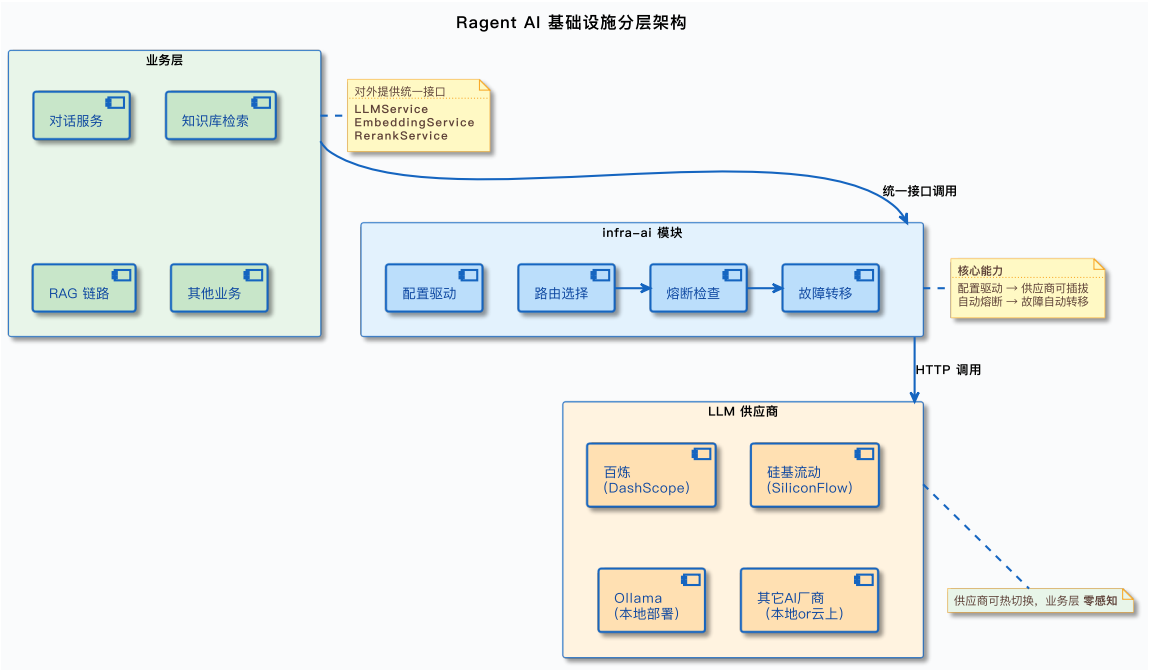
配置驱动，零代码切换。 加一个新供应商、调整模型优先级、切换默认模型——这些操作不应该改代码，改配置文件就够了。

新供应商可插拔扩展。 哪天要接入一个 vLLM 私有部署的推理服务，只需要实现一个客户端类，不需要动任何已有代码。

这就是 AI 基础设施层要解决的问题。

3. infra-ai 模块的定位

用一句话概括：infra-ai 是业务层和模型供应商之间的中间层。



业务层只依赖 infra-ai 暴露的三个接口（LLMService、EmbeddingService、RerankService），完全不感知具体供应商。供应商是谁、优先级怎么排、挂了怎么切——这些全部由 infra-ai 内部处理。

整体架构总览

1. 模块包结构

infra-ai 模块共有 9 个包，每个包有明确的职责边界：

包名	职责	核心类
config	配置根	AIModelProperties——将 YAML 配置绑定为类型安全的 Java 对象
enums	类型词汇表	ModelProvider（供应商枚举）、ModelCapability（能力枚举）
model	路由核心	ModelSelector（选谁）、ModelHealthStore（能不能调）、ModelRoutingExecutor（怎么调）、ModelTarget（调用目标）、ModelCaller（函数式调用接口）
http	HTTP 基础设施	ModelUrlResolver（URL 解析）、HttpResponseHelper（响应工具）、ModelClientException（统一异常）、

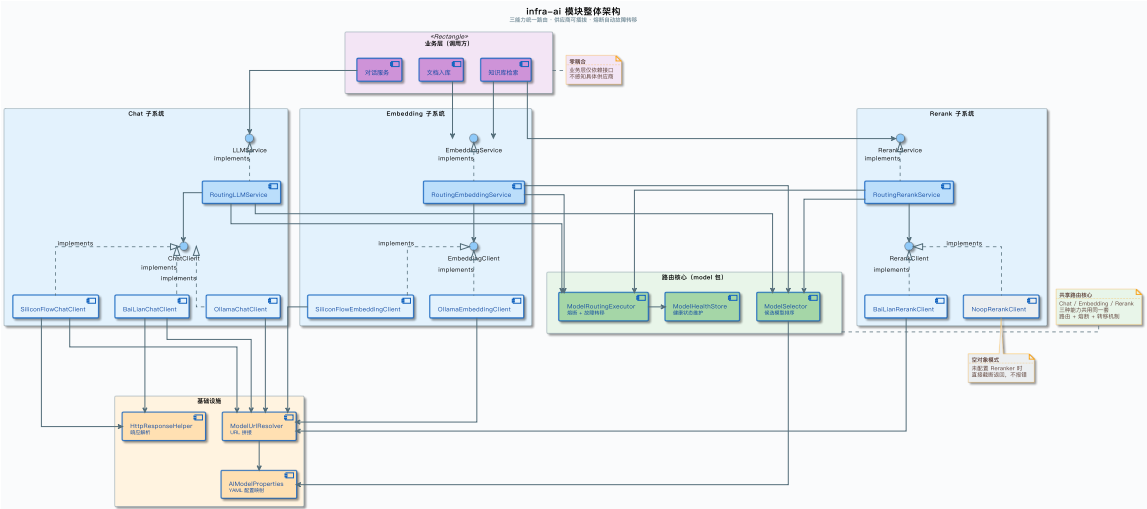
		ModelClientErrorType（错误分类）、HttpMediaTypes（常量）
chat	LLM 对话子系统	LLMService（业务接口）、ChatClient（供应商接口）、AbstractOpenAIStyleChatClient（模板方法基类）、三个供应商实现、RoutingLLMService（路由服务）、流式相关组件
embedding	向量化子系统	EmbeddingService（业务接口）、EmbeddingClient（供应商接口）、两个供应商实现、RoutingEmbeddingService（路由服务）
rerank	重排序子系统	RerankService（业务接口）、RerankClient（供应商接口）、BailianRerankClient（百炼实现）、NoopRerankClient（空对象实现）、RoutingRerankService（路由服务）
token	Token 估算	TokenCounterService（接口）、HeuristicTokenCounterService（启发式实现）
util	工具类	LLMResponseCleaner（清理大模型响应中的 Markdown 代码围栏）

整个模块约 40 个类，代码量不大，但设计上很紧凑。9 个包可以分成三层来理解：

- 底层基础设施：config、enums、http、token、util——提供配置、常量、HTTP 工具等基础能力
- 路由核心：model——提供模型选择、健康检查、故障转移执行器，是整个模块的骨架
- 能力子系统：chat、embedding、rerank——三条并行的业务线，每条线都建立在路由核心之上

2. 三种能力的并行结构

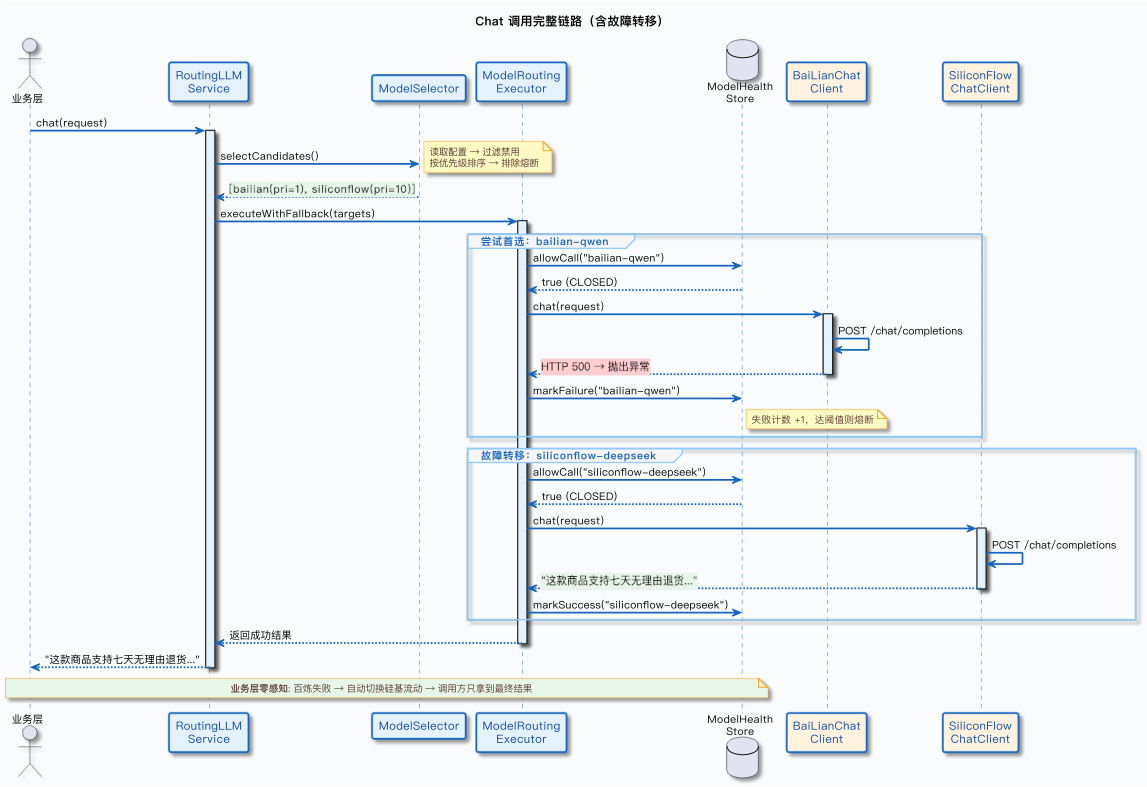
Chat、Embedding、Rerank 三条线的结构高度一致，都遵循同样的三层模式：



关键在于：三种能力子系统共享同一套路由核心（ModelSelector、ModelHealthStore、ModelRoutingExecutor）。这意味着路由逻辑、熔断逻辑、故障转移逻辑只需要写一次，三种能力都能复用。

3. 一次模型调用的完整链路

以一次同步 Chat 调用为例，看看从业务层发起请求到拿到模型响应，中间经过了哪些步骤：



整个链路的核心逻辑是：按优先级逐个尝试候选模型，每次调用前检查熔断器是否放行，调用成功则返回，失败则标记并切换到下一个候选。这个过程对业务层完全透明。

配置驱动的设计

1. 配置结构总览

infra-ai 模块的路由体系完全由一份 YAML 配置驱动。切换供应商、调整优先级、添加新模型——改配置就行，不动代码。
完整配置如下：

```
ai:
  # ===== 供应商配置 =====
  providers:
    ollama:
      url: http://localhost:11434
      # Ollama 不需要 API Key
      endpoints:
        chat: /v1/chat/completions
        embedding: /api/embed
    bailian:
      url: https://dashscope.aliyuncs.com
      api-key: ${BAILIAN_API_KEY:}
      endpoints:
        chat: /compatible-mode/v1/chat/completions
        rerank: /api/v1/services/rerank/text-rerank/text-rerank
    siliconflow:
      url: https://api.siliconflow.cn
      api-key: ${SILICONFLOW_API_KEY:}
      endpoints:
        chat: /v1/chat/completions
        embedding: /v1/embeddings

  # ===== 熔断策略 =====
  selection:
    failure-threshold: 2          # 连续失败 2 次触发熔断
    open-duration-ms: 30000      # 熔断持续 30 秒

  # ===== 流式配置 =====
  stream:
```

```
message-chunk-size: 1 # 流式消息合并块大小

# ===== Chat 模型组 =====
chat:
  default-model: qwen3-max # 默认使用的模型
  deep-thinking-model: qwen3-max # 深度思考模型
  candidates:
    - id: glm-4.7 # 唯一标识
      provider: siliconflow # 关联供应商
      model: Pro/zai-org/GLM-4.7 # 模型名称
      supports-thinking: true # 支持深度思考
      priority: 0 # 优先级最高
    - id: qwen-plus
      provider: bailian
      model: qwen-plus-latest
      priority: 1
    - id: qwen3-local
      provider: ollama
      model: qwen3:8b-fp16
      priority: 2 # 本地模型，中等优先级
    - id: qwen3-max
      provider: bailian
      model: qwen3-max
      supports-thinking: true
      priority: 3

# ===== Embedding 模型组 =====
embedding:
  default-model: qwen-emb-8b
  candidates:
    - id: qwen-emb-8b
      provider: siliconflow
      model: Qwen/Qwen3-Embedding-8B
      dimension: 1024 # 向量维度
      priority: 1
    - id: qwen-emb-local
      provider: ollama
      model: qwen3-embedding:8b-fp16
      dimension: 1024
      priority: 2 # 本地备选

# ===== Rerank 模型组 =====
rerank:
  default-model: qwen3-rerank
  candidates:
    - id: qwen3-rerank
      provider: bailian
      model: qwen3-rerank
      priority: 1
    - id: rerank-noop
      provider: noop # 空实现，兜底用
      model: noop
      priority: 100
```

这份配置看起来信息量不小，拆开来其实就三块：供应商配置、模型组配置、策略配置。

2. 供应商配置 (ProviderConfig)

providers 下面的每个条目定义了一个供应商的连接信息：

```
url: 供应商的基础 URL
api-key: 认证密钥 (Ollama 不需要，所以没配)
endpoints: 按能力类型映射的端点路径
```

为什么要把端点路径放在配置里？因为不同供应商的端点路径不同。拿 Chat 来说，百炼是 `/compatible-mode/v1/chat/completions`，硅基流动和 Ollama 虽然都走 OpenAI 兼容协议，路径都是 `/v1/chat/completions`，但百炼的路径就完全不一样。Embedding 差异更大，Ollama 用的是自己的 `/api/embed`，硅基流动用的是 `/v1/embeddings`。如果把这些路径写死在代码里，每加一个供应商就得改代码。放在配置里，加新供应商只需要多配一个条目。

3. 模型候选配置（ModelCandidate）

chat、embedding、rerank 三个模型组各自维护一个候选列表。每个候选的核心字段：

字段	含义	示例
id	唯一标识，用于路由和日志	qwen-plus
provider	关联到哪个供应商	bailian
model	模型名称，传给供应商 API	qwen-plus-latest
priority	优先级，数字越小越优先	1
enabled	是否启用（默认 true）	true
dimension	向量维度（Embedding 专用）	1536
supports-thinking	是否支持深度思考	false
url	模型级 URL 覆盖（可选）	——

重点是 priority 机制。同一种能力下可以配置多个候选，路由时按优先级从小到大排序。拿 Chat 模型组来看，百炼 qwen3-max 是默认模型，如果默认模型失败，继续优先级 0 的硅基流动 GLM-4.7 排第一，失败了自动切换到优先级 1 的百炼 qwen-plus，再不行还有优先级 2 的本地 Ollama qwen3:8b-fp16 依次兜底。
url 字段是一个候选级的 URL 覆盖。正常情况下，最终调用的 URL 是供应商的 url + endpoints 中对应能力的路径拼接而成。但如果某个候选配置了自己的 url，就直接用它，跳过拼接。这在某些场景下很有用——比如你有一个私有部署的推理服务，URL 和公共供应商不同，单独配一个就行。

4. 熔断策略配置（Selection）

selection 控制熔断器的行为：

- failure-threshold: 2——某个模型连续失败 2 次，触发熔断，后续请求不再尝试它
- open-duration-ms: 30000——熔断持续 30 秒，30 秒后允许一个探测请求试探它是否恢复

熔断器的三态状态机（CLOSED → OPEN → HALF_OPEN）是整个容错体系的核心，后续文章会详细讲实现。这里只需要知道：这两个参数控制的是何时熔断、何时尝试恢复。

5. 流式字段

message-chunk-size 字段是个优化项，如果按照大模型默认的 Token 吞吐量，前端无法优雅实现打字机效果，会显得一片一片出现文字，视觉效果不好。通过这个字段优化体验感。

6. AIModelProperties 类结构

上面那份 YAML 配置，在 Java 侧通过 @ConfigurationProperties(prefix = "ai") 自动绑定到 AIModelProperties 类：

```
@Data
@Configuration
@ConfigurationProperties(prefix = "ai")
public class AIModelProperties {

    private Map<String, ProviderConfig> providers = new HashMap<>();
    private ModelGroup chat = new ModelGroup();
    private ModelGroup embedding = new ModelGroup();
    private ModelGroup rerank = new ModelGroup();
    private Selection selection = new Selection();
    private Stream stream = new Stream();

    @Data
    public static class ProviderConfig {
        private String url;
```

```
private String apiKey;
private Map<String, String> endpoints = new HashMap<>();
}

@Data
public static class ModelGroup {
    private String defaultModel;
    private String deepThinkingModel;
    private List<ModelCandidate> candidates = new ArrayList<>();
}

@Data
public static class ModelCandidate {
    private String id;
    private String provider;
    private String model;
    private String url;
    private Integer dimension;
    private Integer priority = 100;
    private Boolean enabled = true;
    private Boolean supportsThinking = false;
}

@Data
public static class Selection {
    private Integer failureThreshold = 2;
    private Long openDurationMs = 30000L;
}

@Data
public static class Stream {
    private Integer messageChunkSize = 5;
}
}
```

YAML 和 Java 类是一一对应的。providers 映射到 Map<String, ProviderConfig>, chat / embedding / rerank 映射到三个 ModelGroup, 每个 ModelGroup 里有一个 List<ModelCandidate>。整个配置树在应用启动时就绑定好了, 路由组件直接读取, 不需要运行时解析 YAML。

三层接口设计

1. 业务层接口

业务层只需要依赖三个接口, 不需要知道背后的供应商是谁:

```
// LLM 对话接口
public interface LLMService {
    String chat(ChatRequest request);
    StreamCancellationHandle streamChat(ChatRequest request, StreamCallback callback);
}

// 向量化接口
public interface EmbeddingService {
    List<Float> embed(String text);
    List<List<Float>> embedBatch(List<String> texts);
    int dimension();
}

// 重排序接口
public interface RerankService {
    List<RetrievedChunk> rerank(String query, List<RetrievedChunk> candidates, int topN);
}
```

LLMService 支持同步调用 `chat()` 和流式调用 `streamChat()`。流式调用返回一个 `StreamCancellationHandle`，业务层可以随时通过 `handle.cancel()` 取消正在进行的生成。

EmbeddingService 支持单条 `embed()` 和批量 `embedBatch()`，还提供 `dimension()` 方法返回向量维度，用于向量库 Schema 定义。

RerankService 最简单，就一个 `rerank()` 方法，传入 `query` 和候选文档列表，返回精排后的 topN 结果。

这三个接口还提供了一些便捷的默认方法。比如 LLMService 有 `chat(String prompt)` 的简化版本，内部自动构造 `ChatRequest`：

```
default String chat(String prompt) {
    ChatRequest req = ChatRequest.builder()
        .messages(List.of(ChatMessage.user(prompt)))
        .build();
    return chat(req);
}
```

简单问答场景一行代码就够了：`llmService.chat("什么是 RAG? ")`。

2. 供应商接口

供应商接口是面向具体模型供应商的，每个供应商实现一套：

```
// Chat 供应商接口
public interface ChatClient {
    String provider();
    String chat(ChatRequest request, ModelTarget target);
    StreamCancellationHandle streamChat(ChatRequest request, StreamCallback callback, ModelTarget target);
}

// Embedding 供应商接口
public interface EmbeddingClient {
    String provider();
    List<Float> embed(String text, ModelTarget target);
    List<List<Float>> embedBatch(List<String> texts, ModelTarget target);
}

// Rerank 供应商接口
public interface RerankClient {
    String provider();
    List<RetrievedChunk> rerank(String query, List<RetrievedChunk> candidates, int topN, ModelTarget target);
}
```

和业务层接口相比，供应商接口有两个明显的区别：

第一，多了一个 `provider()` 方法，返回供应商标识（比如 "bailian"、"siliconflow"、"ollama"）。这个方法用于注册——路由服务在启动时会收集所有供应商客户端，按 `provider()` 建立索引，调用时根据候选模型的 `provider` 字段查找对应的客户端。

第二，多了一个 `ModelTarget` 参数。`ModelTarget` 是一个 record，把候选配置和供应商配置打包在一起：

```
public record ModelTarget(
    String id,
    AIModelProperties.ModelCandidate candidate,
    AIModelProperties.ProviderConfig provider
) {}
```

为什么需要这个参数？因为供应商客户端需要知道具体调哪个模型（`candidate.getModel()`）、用哪个 URL（`provider.getUrl()`）、带什么 API Key（`provider.getApiKey()`）。这些信息在路由阶段确定，通过 `ModelTarget` 传递给供应商客户端。

2.1 为什么分两层接口

业务层接口负责路由决策——选哪个模型、失败了怎么切换。供应商接口负责具体执行——构建 HTTP 请求、解析响应、处理流式数据。职责不同，所以分开。业务层的调用者不需要关心路由逻辑，供应商的实现者也不需要关心路由逻辑，各管各的。如果把路由和执行混在一起，每个供应商实现里都得写一遍路由逻辑，和前面没有 infra 层的反面例子一样。

3. 路由实现

三个路由服务——RoutingLLMService、RoutingEmbeddingService、RoutingRerankService——是业务层接口的 @Primary 实现。它们是连接业务层接口和供应商接口的桥梁。

路由服务的构造过程是这样的：Spring 容器启动时，通过 List<ChatClient> 注入所有 ChatClient 的实现（BaiLianChatClient、SiliconFlowChatClient、OllamaChatClient），路由服务在构造函数里遍历这个列表，按 provider() 方法的返回值建立 Map<String, ChatClient> 索引。

调用时，路由服务委托 ModelSelector 获取按优先级排序的候选列表，再通过 ModelRoutingExecutor 逐个尝试。执行器拿到候选的 provider 字段，从 Map 里查找对应的客户端，然后调用它。

三个路由服务的结构高度一致，说明这套设计的复用性很好。差异只在于：RoutingLLMService 的流式调用需要特殊处理（首包探测机制），其余都是标准的 executeWithFallback 委托。

路由核心：四个组件

model 包是整个 infra-ai 模块的骨架，包含四个核心组件。

1. ModelTarget——调用目标

```
public record ModelTarget(  
    String id,  
    AIModelProperties.ModelCandidate candidate,  
    AIModelProperties.ProviderConfig provider  
) {}
```

ModelTarget 是一个不可变的运行时数据包。它把候选配置（模型名、优先级、维度等）和供应商配置（URL、API Key、端点路径等）打包在一起，作为一次模型调用的完整上下文传递给供应商客户端。

为什么要用 record 而不是一个普通的 POJO？因为它是不可变的、自动实现了 equals / hashCode / toString，非常适合作为数据载体在组件之间传递。

2. ModelSelector——选谁

ModelSelector 是模型选择器，职责是从配置中选出当前可用的候选模型列表。它的工作流程：

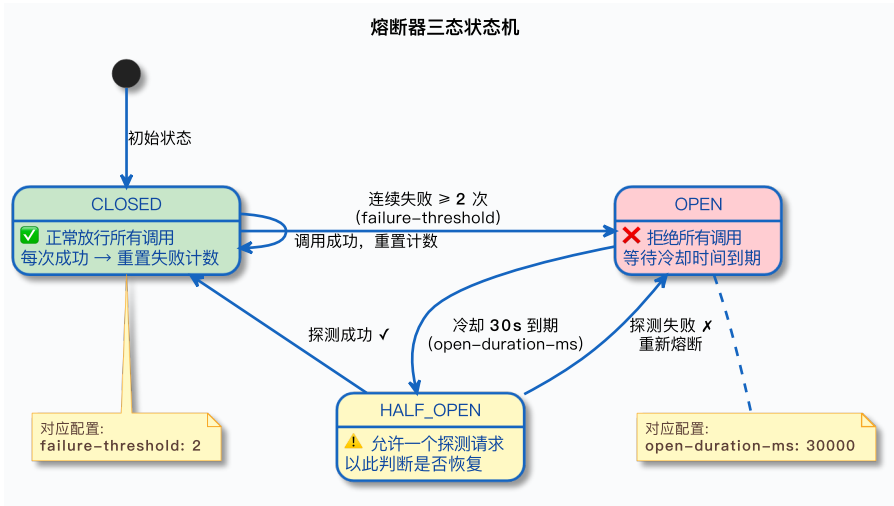
1. 从 AIModelProperties 中读取对应能力（Chat / Embedding / Rerank）的候选列表
2. 过滤掉 enabled = false 的候选
3. 按 priority 从小到大排序
4. 如果配置了 defaultModel，把它提升到列表第一位
5. 排除被熔断器标记为不可用的模型
6. 返回有序的 List<ModelTarget>

Chat 能力额外支持深度思考模式的过滤——如果调用方指定了深度思考，ModelSelector 会把 deepThinkingModel 提到首位，同时过滤出 supportsThinking = true 的候选。

3. ModelHealthStore——能不能调

ModelHealthStore 是一个三态熔断器，用于保护系统不会持续调用已经故障的模型。

三个状态之间的转换逻辑：



CLOSED（正常）：所有调用放行。每次调用成功，失败计数重置为 0。每次失败，失败计数 +1。

OPEN（熔断）：拒绝所有调用。当连续失败次数达到 `failureThreshold`（默认 2 次），状态从 CLOSED 切换到 OPEN，持续 `openDurationMs`（默认 30 秒）。

HALF_OPEN（半开）：冷却时间过后，状态切换到 HALF_OPEN，允许一个探测请求通过。如果探测成功，回到 CLOSED；如果失败，重新回到 OPEN。

这里只讲概念，不展示代码。后续文章会详细讲 `ModelHealthStore` 如何用 `ConcurrentHashMap.compute()` 保证并发安全的状态转换。

4. ModelRoutingExecutor——怎么调

`ModelRoutingExecutor` 是通用的故障转移执行器。它接收四个参数：

- `capability`：能力类型（用于日志）
- `targets`：候选模型列表（已排序）
- `clientResolver`：根据 `ModelTarget` 查找对应客户端的函数
- `caller`：实际调用函数（通过 `ModelCaller` 函数式接口传入）

执行逻辑（伪代码）：

```
for (Target target : targets) {
    Client client = clientResolver.resolve(target);
    if (client == null) {
        continue;
    }

    if (!healthStore.allowCall(target.getId())) {
        continue;
    }

    try {
        Result result = caller.call(client, target);
        healthStore.markSuccess(target.getId());
        return result;
    } catch (Exception e) {
        healthStore.markFailure(target.getId());
        log.warn("调用目标 [{}] 失败: {}", target.getId(), e.getMessage());
    }
}

throw new RemoteException("所有候选模型均调用失败");
```

它是一个泛型方法，通过 `ModelCaller<C, T>` 函数式接口支持任意客户端类型和返回类型。`Chat`、`Embedding`、`Rerank` 三种能力共用同一个执行器：

```
@FunctionalInterface
public interface ModelCaller<C, T> {
    T call(C client, ModelTarget target) throws Exception;
}
```

`Chat` 调用时，`C` 是 `ChatClient`，`T` 是 `String`。`Embedding` 调用时，`C` 是 `EmbeddingClient`，`T` 是 `List<Float>`。泛型让一个执行器适配所有场景。

5. 四个组件如何协作

把四个组件串起来：

- 业务层调用 `LLMService.chat(request)`
- `RoutingLLMService` 调用 `ModelSelector.selectChatCandidates()` 获取候选列表，返回的就是一组 `ModelTarget`
- `RoutingLLMService` 把候选列表传给 `ModelRoutingExecutor.executeWithFallback()`
- 执行器遍历候选列表，对每个 `ModelTarget`：

- 调用 `ModelHealthStore.allowCall()` 检查熔断器是否放行
- 如果放行，通过 `ModelCaller` 调用对应的 `ChatClient`
- 成功则 `markSuccess()` 并返回结果
- 失败则 `markFailure()` 并切换到下一个候选

5. 所有候选都失败，抛出 `RemoteException`

`ModelTarget` 是数据载体，`ModelSelector` 负责选谁，`ModelHealthStore` 负责判断能不能调，`ModelRoutingExecutor` 负责执行和切换。四个组件各有分工，组合起来就是一个完整的带故障转移的模型调用流程。

核心设计模式一览

整个 `infra-ai` 模块用到了多种设计模式，它们不是为了炫技，而是 AI 调度场景下自然产生的需求：

模式	应用位置	解决的问题
策略模式	<code>ChatClient</code> / <code>EmbeddingClient</code> / <code>RerankClient</code>	供应商可插拔，新增供应商只需实现接口，不改已有代码
模板方法	<code>AbstractOpenAIStyleChatClient</code>	三个 Chat 供应商都兼容 OpenAI 协议，请求构建、响应解析、流式处理的通用逻辑提到基类，子类只关注差异点（是否需要 API Key、是否有特殊请求字段）
注册表 / 工厂	<code>RoutingLLMService</code> 等路由服务	Spring 自动注入所有客户端实现，按 <code>provider()</code> 建立 Map 索引，运行时按供应商名查找。加新供应商只需注册一个 Bean
熔断器	<code>ModelHealthStore</code>	避免持续调用已故障的模型，给它恢复时间，防止故障级联
故障转移链	<code>ModelRoutingExecutor.executeWithFallback()</code>	按优先级逐个尝试候选模型，当前失败自动切换下一个，对业务层完全透明
装饰器	<code>ProbeBufferingCallback</code>	流式调用首包探测阶段缓冲所有事件，探测成功后再刷出给真实回调，避免错误数据推送到前端
函数式接口	<code>ModelCaller<C, T></code>	让通用执行器 <code>executeWithFallback</code> 支持任意客户端类型和返回类型，一个方法适配 Chat / Embedding / Rerank

这些模式不是孤立使用的，而是相互配合。策略模式让供应商可插拔，注册表模式让 Spring 自动发现它们，模板方法让同协议的供应商复用代码，熔断器和故障转移链让调用链路具备容错能力。后续文章在讲到具体实现时，会结合代码详细拆解每种模式的用法。

流式调用的特殊处理（预告）

前面讲的调用链路是同步调用的情况，逻辑比较直接：调用 → 拿返回值 → 判断成功或失败。但流式调用要复杂得多。同步调用可以通过返回值判断是否成功——HTTP 200 且响应体有内容就是成功。但流式调用是异步的，`client.streamChat()` 调用后立即返回一个取消句柄，真正的数据通过 `StreamCallback` 回调推送。问题在于：调用返回的那一刻，你并不知道这个供应商是否真的能正常工作。可能连接建立了但供应商迟迟不返回内容，也可能返回了一个错误。如果不做探测就直接把回调交给真实的前端 SSE 连接，一旦这个供应商有问题，用户看到的就是要么一直 loading，要么收到一个错误——而此时想切换到备选供应商已经来不及了，因为前端的 SSE 连接已经绑定了。`RoutingLLMService` 用 `ProbeBufferingCallback` + `FirstPacketAwaiter` 实现了一个 `probe-and-commit` 机制：先用一个缓冲装饰器包装真实回调，等待第一个 token 到达（最多 60 秒），确认供应商可用后再把缓冲的事件刷出给真实回调。如果超时或出错，取消当前连接，切换到下一个候选重试。这是整个模块最复杂的部分，后续文章会单独详细讲解。

小结与下一步

回顾一下这篇文章的核心要点：

- `infra-ai` 模块是业务层和模型供应商之间的中间层，解决统一接口、多模型路由、故障转移、配置驱动四个核心问题
- 9 个包按职责清晰划分：底层基础设施（`config` / `enums` / `http` / `token` / `util`）、路由核心（`model`）、能力子系统（`chat` / `embedding` / `rerank`）
- 配置驱动设计：一份 YAML 配置驱动整个路由体系，供应商配置和模型候选配置分离，切换模型只需改配置

三层接口设计：业务层接口（LLMService / EmbeddingService / RerankService）→ 路由实现（RoutingXxxService）→ 供应商接口（ChatClient / EmbeddingClient / RerankClient），职责清晰

路由核心四个组件各有分工：ModelSelector 选谁、ModelHealthStore 能不能调、ModelRoutingExecutor 怎么调、ModelTarget 封装调用上下文

多种设计模式协同工作：策略模式实现供应商可插拔、模板方法复用 OpenAI 兼容协议、注册表实现自动发现、熔断器提供容错保护

这一篇讲的是宏观设计，建立全局视角。接下来我们深入路由核心，拆解**多模型路由与智能选择**——ModelSelector 的优先级排序算法、首选模型提升机制、深度思考模型的过滤逻辑、禁用和熔断候选的过滤策略，以及 ModelTarget 的完整构建过程。

